API 自动化测试框架代码生成规范 v3.0

📋 版本说明

版本: v3.0

更新日期: 2026-02-04

基于框架版本: 含 Loguru、.env、DAO、失败重试

重大变更: 全面替换 print() 为 logger，引入环境配置管理

---

1. 项目架构与目录结构

```
api_auto_framework/
├── api/                    # API 封装层
│   ├── __init__.py
│   └── {module}_api.py     # 各模块 API 类
├── config/                 # 配置管理
│   ├── __init__.py
│   └── config.py           # 多环境配置（支持 .env）
├── dao/                    # 数据库操作层 ⭐新增
│   ├── __init__.py
│   └── db_manager.py       # 数据库连接池管理
├── data/                   # 测试数据与枚举
│   ├── __init__.py
│   └── enums.py            # 枚举类型定义
├── logs/                   # 日志文件目录 ⭐新增
│   └── test_{date}.log     # 按天切割日志
├── test_cases/             # 测试用例
│   ├── conftest.py         # 全局 fixture 与钩子
│   ├── {module}/           # 按模块组织
│   │   ├── conftest.py     # 模块级 fixture
│   │   └── test_*.py       # 测试文件
├── utils/                  # 工具函数
│   ├── __init__.py
│   ├── assertions.py       # 断言助手
│   └── logger.py           # 日志配置 ⭐新增
├── assets/                 # 报告模板
│   └── report_template.html
├── reports/                # 测试报告
├── .env                    # 环境配置（git ignore）⭐新增
├── .env.example            # 配置模板 ⭐新增
├── .gitignore              # Git 忽略规则 ⭐新增
├── pytest.ini              # pytest 配置（含重试）⭐更新
├── requirements.txt        # 依赖清单
└── README.md               # 项目说明
```

---

2. 命名规范

2.1 文件命名

类型	格式	示例
API 封装	`{module}_api.py`	`account_api.py`
测试文件	`test_{module}_{action}.py`	`test_account_list.py`
配置文件	`conftest.py`	模块级/全局
工具模块	`{function}.py`	`assertions.py`

2.2 类命名

类型	格式	示例
API 类	`{Module}API`	`AccountAPI`
测试类	`Test{Module}{Action}`	`TestAccountList`
枚举类	`{Name}`	`AccountStatus`

2.3 函数/方法命名

类型	格式	示例
测试方法	`test_{action}_{scenario}`	`test_list_accounts_with_filter`
API 方法	`{action}_{resource}`	`list_accounts`
fixture	`{module}_api`	`account_api`

---

3. API 封装层规范

3.1 文件模板

```python
"""
{模块名称} API 封装
遵循 API Object 模式，提供灵活的参数化接口
"""
import requests
from typing import Optional, Union, Dict, Any
from config.config import config
from data.enums import {相关枚举}
from utils.logger import logger  # ⭐必需


class {Module}API:
    """
    {模块名称} API 封装类
    """
    def __init__(self, session: Optional[requests.Session] = None):
        """
        初始化 {Module}API

        Args:
            session: requests.Session 对象，如果传入则使用该 session 保持认证状态
        """
        self.base_url = config.base_url
        self.config = config
        self.session = session or requests.Session()

    def {action}_{resource}(self, ...) -> requests.Response:
        """
        {接口功能描述}

        Args:
            ...: 参数说明

        Returns:
            requests.Response: 响应对象
        """
        url = self.config.get_full_url("/{endpoint}")

        # ⭐ 使用 logger 替代 print
        logger.debug(f"请求 {url}，参数: {params}")

        response = self.session.{method}(url, ...)
```

```python

        logger.debug(f"响应状态: {response.status_code}")

        return response

    def parse_{type}_response(self, response: requests.Response) -> dict:
        """
        解析响应，提取关键信息

        Args:
            response: 原始响应对象

        Returns:
            dict: 包含 error, message, data 等字段
        """
        if response.status_code != 200:
            logger.warning(f"非200状态码: {response.status_code}")
            return {
                "error": True,
                "message": f"HTTP {response.status_code}",
                "data": None
            }

        try:
            data = response.json()
            logger.debug(f"解析成功: {len(data.get('content', []))} 条数据")
            return {
                "error": False,
                "message": "Success",
                "data": data
            }
        except Exception as e:
            logger.error(f"解析失败: {e}", exc_info=True)
            return {
                "error": True,
                "message": str(e),
                "data": None
            }
```

3.2 关键规则

1. **必需导入**: `from utils.logger import logger`

2. **日志替代 print**:
   - 请求参数: `logger.debug(f"请求参数: {params}")`
   - 响应状态: `logger.debug(f"响应状态: {status_code}")`
   - 错误异常: `logger.error(f"请求失败: {e}", exc_info=True)`
3. **使用 config.get_full_url()** 构建完整 URL
4. **枚举转换**: 支持 `Union[str, Enum]` 类型参数

---

4. 测试用例规范

4.1 文件模板

```python
"""
{模块名称} 接口测试用例
测试 {HTTP_METHOD} /api/v1/cores/{core}/{endpoint} 接口
"""
import pytest
from utils.logger import logger  # ⭐必需
from utils.assertions import (
    assert_status_ok,
    assert_response_parsed,
    assert_list_structure,
    assert_fields_present
)


@pytest.mark.{module}
@pytest.mark.{api_type}
class Test{Module}{Action}:
    """
    {模块名称} {动作} 接口测试用例集
    """

    def test_{action}_{scenario}(self, {module}_api):
        """
        测试场景{N}: {中文描述}
        验证点:
        1. {验证点1}
        2. {验证点2}
        """
        # 1. 准备测试数据
        logger.info("测试场景{N}: {描述}")

        # 2. 调用 API
        logger.info("调用 {API名称} 接口")
        response = {module}_api.{method}(...)

        # 3. 验证结果
        logger.info("验证响应状态")
        assert_status_ok(response)

        response_body = response.json()
        logger.debug(f"响应数据: {response_body}")

```

```
        # 4. 详细断言
        assert_list_structure(response_body)

        logger.info("√ 测试通过")
```

4.2 日志使用规范 ⭐重点

场景	日志级别	示例
测试开始	`logger.info()`	`logger.info("测试场景1: 成功获取列表")`
关键步骤	`logger.info()`	`logger.info("调用 List API")`
详细数据	`logger.debug()`	`logger.debug(f"响应数据: {data}")`
预期错误	`logger.warning()`	`logger.warning(f"业务错误: {code}")`
测试通过	`logger.info()`	`logger.info("√ 测试通过")`

⚠️ **严禁使用 print()，所有输出必须使用 logger**

4.3 Docstring 规范

```
def test_list_accounts_with_name_filter(self, account_api):
    """
    测试场景2: 名称筛选查询 - 验证筛选逻辑
    验证点:
    1. 接口返回 200
    2. 返回结果中所有账户名称都包含筛选关键词
    3. 分页信息正确
    """
```

格式要求:

- 第一行：`测试场景{N}：{中文描述}`（N 为递增数字）
- 第二行：`验证点：`（可选）
- 后续行：具体验证点列表

---

5. 断言助手规范

5.1 使用规范

```python
from utils.assertions import (
    assert_status_ok,          # HTTP 状态码验证
    assert_response_parsed,    # 响应解析验证
    assert_list_structure,     # 列表结构验证
    assert_enum_filter,        # 枚举筛选验证
    assert_string_contains_filter,  # 字符串包含验证
    assert_pagination,         # 分页验证
    assert_empty_result,       # 空结果验证
    assert_fields_present      # 字段存在性验证
)

# 使用示例
assert_status_ok(response, 200)
assert_list_structure(response_body)
assert_fields_present(item, ["id", "name", "status"])
```

5.2 断言函数内部日志 ⭐已升级

所有断言函数已自动包含日志：

- **成功**: `logger.debug("断言通过: ...")`
- **失败**: `logger.error("断言失败: ...")` + `raise AssertionError`

开发者无需手动添加断言日志。

---

6. Fixture 规范

6.1 全局 Fixture (test_cases/conftest.py)

```python
from utils.logger import logger  # ⭐必需
from dao.db_manager import DBManager  # ⭐新增

@pytest.fixture(scope="session", autouse=True)
def login_session():
    """全局登录会话管理"""
    session = requests.Session()
    logger.info("正在执行全局登录...")  # ⭐替代 print

    # 登录逻辑...

    yield session
    logger.info("正在关闭会话...")  # ⭐替代 print
    session.close()


@pytest.fixture(scope="session")  # ⭐新增
def db_session():
    """数据库会话管理（可选）"""
    db_manager = None
    try:
        db_config = config.db_config
        db_manager = DBManager(db_config)
        logger.info("数据库会话已就绪")
        yield db_manager
    except Exception as e:
        logger.warning(f"数据库会话初始化跳过: {e}")
        yield None
    finally:
        if db_manager:
            db_manager.close_connection()
```

6.2 模块级 Fixture (test_cases/{module}/conftest.py)

```python
import pytest
from api.{module}_api import {Module}API

pytestmark = pytest.mark.{module}


@pytest.fixture(scope="module")
def {module}_api(login_session):
    """
    {模块名称} API fixture
    """
    return {Module}API(session=login_session)
```

---

7. 配置管理规范 ⭐新增

7.1 环境变量文件 (.env)

示例:

```env
# 环境标识
ENV=dev

# API 配置
BASE_URL=https://api-dev.accelerationcloud.info

# 数据库配置
DB_HOST=localhost
DB_PORT=5432
DB_USER=test_user
DB_PASSWORD=secure_password
DB_NAME=test_db

# 认证配置（可选）
TENANT_ID=your_tenant_id
USER_ID=your_user_id
BASIC_AUTH=your_encoded_token
```

7.2 配置读取

```python
from config.config import config

# 获取环境
env = config.get_env()  # "DEV" / "UAT"

# 获取 URL
base_url = config.base_url

# 获取数据库配置
db_config = config.db_config
```

优先级: `.env 文件` > `代码默认值`

---

8. 数据库操作规范 ⭐新增

8.1 使用场景

- ✅ 验证 API 操作后的数据库状态
- ✅ 为测试准备特定数据状态
- ✅ 测试后清理数据
- ❌ 纯 API 契约验证（不需要数据库）

8.2 使用示例

```python
def test_account_create_with_db_verification(account_api, db_session):
    """
    测试场景1: 创建账户并验证数据库落库
    """
    logger.info("测试场景1: 创建账户并验证数据库")

    # 1. 创建账户
    response = account_api.create_account({...})
    account_id = response.json()["id"]
    logger.info(f"账户创建成功: {account_id}")

    # 2. 验证数据库（可选）
    if db_session:
        sql = "SELECT * FROM accounts WHERE id = :id"
        result = db_session.execute_query(sql, {"id": account_id})

        assert len(result) == 1, "数据未落库"
        logger.info("√ 数据库验证通过")
```

---

9. pytest 配置规范 ⭐更新

9.1 pytest.ini 配置

```
[pytest]
testpaths = test_cases
python_files = test_*.py
python_classes = Test*
python_functions = test_*

addopts = -v -s

# 失败重试配置 ⭐新增
reruns = 2
reruns_delay = 1

markers =
    {module}: {模块} 模块相关测试
    {api_type}: {类型} 接口测试
    no_rerun: 禁止失败重试的测试 ⭐新增

filterwarnings =
    ignore::urllib3.exceptions.NotOpenSSLWarning
```

9.2 重试标记使用

```
# 允许重试（默认）
def test_list_accounts(account_api):
    """网络波动可自动重试"""
    pass


# 禁止重试 ⭐使用场景
@pytest.mark.no_rerun
def test_delete_account(account_api):
    """破坏性操作，禁止重试"""
    pass

@pytest.mark.no_rerun
def test_account_idempotency(account_api):
    """幂等性验证，禁止重试"""
    pass
```

禁止重试场景:

- 破坏性操作（delete, logout, change_password）
- 幂等性验证
- 断言失败（业务逻辑错误）

---

10. 测试数据规范

10.1 命名规范

所有自动化生成的测试数据必须包含前缀: `Auto TestYan`

```
contact_data = {
    "first_name": "Auto TestYan",
    "last_name": f"Contact {random_string}",
    "email": f"auto.testyan.{random_string}@example.com"
}

account_data = {
    "account_name": f"Auto TestYan Account {timestamp}",
    "business_name": f"Auto TestYan Business {random_string}"
}
```

10.2 数据清理

```python
@pytest.fixture
def create_test_account(account_api):
    """创建测试账户（自动清理）"""
    created_ids = []

    def _create(**kwargs):
        response = account_api.create_account(kwargs)
        account_id = response.json()["id"]
        created_ids.append(account_id)
        logger.info(f"创建测试账户: {account_id}")
        return response

    yield _create

    # 清理
    for account_id in created_ids:
        try:
            account_api.delete_account(account_id)
            logger.info(f"清理测试账户: {account_id}")
        except Exception as e:
            logger.warning(f"清理失败: {e}")
```

---

11. 报告规范

11.1 报告生成

- **自动生成**: 通过 `conftest.py` 的 `pytest_sessionfinish` 钩子
- **格式**: HTML（自定义模板）
- **命名**: `benxi_report_{timestamp}.html`
- **内容**: 中英文双语，支持切换

11.2 报告数据

```python
# test_cases/conftest.py
def pytest_runtest_makereport(item, call):
    """捕获测试结果与元数据"""
    test_results.append({
        "nodeid": item.nodeid,
        "status": report.outcome,
        "module": module_name,
        "test_file": test_file,
        "docstring_summary_zh": docstring_summary_zh,
        "docstring_summary_en": docstring_summary_en,
        "scenario_number": scenario_number,  # 用于排序
        # ...
    })
```

---

12. Git 提交规范

12.1 提交消息格式

```
{type}: {简要描述}

## {详细说明}（可选）

{多行说明}
```

Type 类型:

- `feat` : 新功能
- `fix` : Bug 修复
- `refactor` : 重构
- `test` : 测试相关
- `docs` : 文档更新
- `chore` : 构建/工具配置

12.2 提交示例

```
git add .
git commit -m "feat: 新增 Account Opening 模块自动化脚本

- 实现 7 个接口封装
- 创建 15 个测试场景
- 支持企业/个人申请流程"

git push origin main
```

---

13. 依赖管理

13.1 requirements.txt 标准

```
# Testing Framework
pytest>=7.4.0
pytest-html>=3.2.0
pytest-json-report>=1.5.0
pytest-rerunfailures>=12.0  # ⭐失败重试

# HTTP Requests
requests>=2.31.0

# Logging ⭐新增
loguru>=0.7.0

# Environment Management ⭐新增
python-dotenv>=1.0.0

# Database
psycopg2-binary>=2.9.0
sqlalchemy>=2.0.0  # ⭐ORM & 连接池
pymysql>=1.1.0     # ⭐MySQL 驱动

# Data Processing
pandas>=2.0.0
openpyxl>=3.1.0
```

13.2 安装命令

```
pip install -r requirements.txt
```

---

14. 开发工作流

14.1 新模块开发流程

1. **创建 API 封装**:

```
touch api/{module}_api.py
# 按模板编写 API 类（使用 logger）
```

2. **创建模块配置**:

```
mkdir test_cases/{module}
touch test_cases/{module}/conftest.py
# 添加 pytestmark 和 {module}_api fixture
```

3. **编写测试用例**:

```
touch test_cases/{module}/test_{module}_{action}.py
# 按规范编写测试类和方法（使用 logger）
```

4. **更新配置**:
   - `pytest.ini`：添加 marker
   - `test_cases/conftest.py`：更新 `MODULE_NAME_MAPPING`
5. **本地测试**:

```
pytest test_cases/{module} -v
# 检查 logs/test_{date}.log
```

6. **提交代码**:

```
git add .
git commit -m "feat: 新增 {module} 模块"
git push origin main
```

14.2 调试技巧

```
# 查看日志文件
tail -f logs/test_$(date +%Y-%m-%d).log

# 运行单个测试
pytest test_cases/module/test_file.py::TestClass::test_method -v -s

# 查看详细日志
pytest --log-cli-level=DEBUG

# 禁用重试（调试时）
pytest --reruns=0
```

---

15. 核心升级变更总结 ⭐v3.0

15.1 必需变更

项目	v2.0 (旧)	v3.0 (新)
输出	`print()`	`logger.info/debug/error()` ⭐
配置	硬编码	`.env` + `config.py` ⭐
重试	无	`pytest.ini` : `reruns=2` ⭐
数据库	无	`dao/db_manager.py` ⭐
日志文件	无	`logs/test_{date}.log` ⭐

15.2 导入变更

旧代码 (v2.0):

```
import pytest
from api.account_api import AccountAPI
```

新代码 (v3.0):

```
import pytest
from utils.logger import logger  # ⭐必需添加
from api.account_api import AccountAPI
```

15.3 输出变更

旧代码 (v2.0):

```
print("\n[Step] 调用 List API")
print(f"返回数据: {response.json()}")
```

新代码 (v3.0):

```
logger.info("调用 List API")  # ⭐无需 \n 和 [Step]
logger.debug(f"返回数据: {response.json()}")  # ⭐详细数据用 debug
```

---

16. 检查清单

16.1 新模块开发检查

- [ ] API 类导入 `from utils.logger import logger`
- [ ] 所有 `print()` 替换为 `logger.*`
- [ ] 测试类导入 `from utils.logger import logger`
- [ ] Docstring 格式正确（含 `测试场景{N}`）

- [ ] 测试数据包含 `Auto TestYan` 前缀
- [ ] fixture 在 `conftest.py` 中定义
- [ ] `pytest.ini` 添加 marker
- [ ] `conftest.py` 更新 `MODULE_NAME_MAPPING`
- [ ] 破坏性测试标记 `@pytest.mark.no_rerun`

16.2 代码质量检查

```
# 检查是否还有 print
grep -r "print(" test_cases/ api/ utils/

# 检查日志文件生成
ls -la logs/

# 运行 linter
pylint test_cases/{module}/ --disable=C0111

# 运行测试覆盖率
pytest --cov=api --cov-report=html
```

---

17. 常见问题 FAQ

Q1: 何时使用 logger.info vs logger.debug?

A:

- `info`：测试步骤、结果摘要（用户关心）
- `debug`：详细参数、响应数据（调试用）

Q2: 数据库 fixture 何时使用?

A:

- ✅ 需要验证数据落库
- ✅ 需要准备复杂数据状态

- ❌ 纯 API 功能测试

Q3: 如何禁用失败重试?

A:

```
@pytest.mark.no_rerun
def test_delete_operation(api):
    ...
```

Q4: .env 文件为什么不提交?

A:

- 包含敏感信息（密码、token）
- 每个开发者配置不同
- 使用 `.env.example` 作为模板

Q5: 日志文件如何清理?

A:

- 自动保留 7 天（`logger.py` 配置）
- 手动清理: `rm logs/*.log.zip`

📚 参考文档

- **框架文档**: `README.md`
- **API 文档**: https://api-docs.accelerationcloud.info
- **Loguru 文档**: https://loguru.readthedocs.io
- **pytest 文档**: https://docs.pytest.org

规范版本: v3.0

生效日期: 2026-02-04

维护者: QA Team yan